STUDENT PROJECT BRIEF · V1.0 (LITE)

NCRTC Bus Management System

A friendly, scoped-down spec for a final-year capstone. Four modules at MVP depth, working with seeded dummy data, using a beginner-friendly stack. You will learn a lot. You will also sleep.

AVLS | Scheduling | IMS | CMS | Seeded data | React + FastAPI/Express

FOR
Final-year B.Tech CS — NSUT Delhi · Team of 4

DURATION
Two semesters

READING TIME
~30 minutes

STATUS
Approved for student build

§1 What You're Building, in One Page

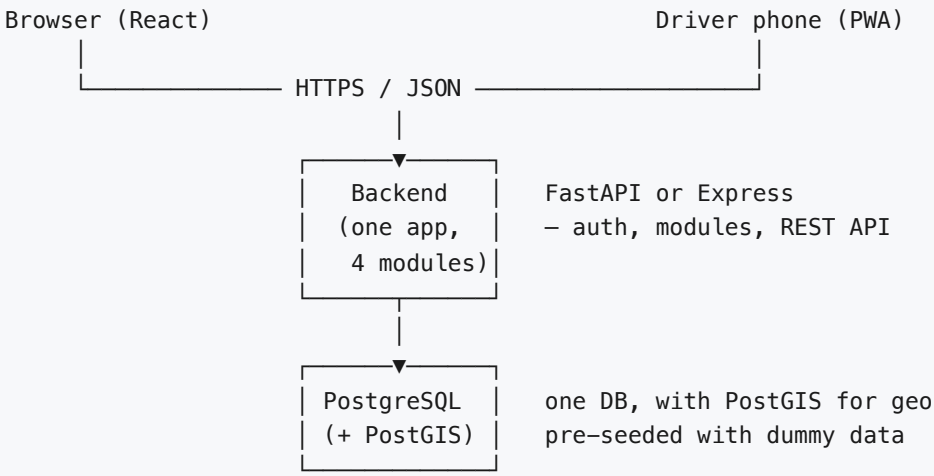
NCRTC runs the feeder bus network around the Delhi-NCR rapid-rail corridor. They want a software system that helps them **see where the buses are**, **plan who drives which bus**, **track problems when they happen**, and **push notices to drivers**. You're building that system as a capstone project.

To keep the scope humane, the project works with **seeded dummy data** — not real GPS hardware, not a high-throughput streaming pipeline. You'll load a few hundred fake vehicles, fake drivers, fake routes, and a sample day of GPS pings. A small background script can drip-feed new pings every few seconds so the live map feels alive. That's it. The shape of the system stays real; the input is simulated.

The four modules

MODULE	WHAT IT DOES	WHO USES IT
AVLS	Live map of all buses + history view of past trips	Control room, depot managers
Scheduling	Create routes; assign drivers to vehicles for the day	Depot managers, drivers
IMS	Raise and resolve incidents (breakdown, complaint, etc.)	Anyone can raise; managers triage
CMS	Publish notices that show up on the driver app	Admin publishes; drivers read

The one-page system shape



> **GOOD NEWS**
>
> You don't need: a message queue, Redis, a microservices mesh, Kubernetes, AWS, a TCP socket layer, or a GPS device protocol. One backend, one database, one frontend, one driver app. Build that well.

§2 About NCRTC (Just Enough Context)

The **National Capital Region Transport Corporation** is a joint venture of the central government and four state governments (Delhi, Haryana, Rajasthan, UP). They run the RRTS rapid-rail line between Delhi and cities like Meerut. Around each rail station, a fleet of feeder buses takes passengers the last few kilometres. That bus fleet is the world your system lives in.

You don't need to know transit-industry trivia. You just need to know that:

- There are several **depots** — places where buses park and crews start their day.
- Each **vehicle** belongs to a depot.
- A **route** is an ordered list of stops with timings.
- A **duty** is a driver's day's work: one driver + one vehicle + one or more trips along a route.
- A **trip** is one end-to-end run of a route by a vehicle.
- An **incident** is anything that went wrong: breakdown, accident, complaint, no-show.

Reread that list once. It's the entire domain. Everything else is just rules and views over those nouns.

§3 Who Uses the System (5 Roles)

ROLE	WHERE THEY WORK	WHAT THEY WANT
Driver	On a phone, anywhere	See today's duty; acknowledge it; tap a panic button if something goes wrong; read notices.
Depot manager	Office laptop	Assign drivers to vehicles for the week; see their depot's buses on the map; handle incidents from their depot.
Control-room operator	Big desktop	See *every* bus on a live map; respond to alerts; assign serious incidents.
Admin	Office laptop	Add new vehicles, drivers, routes, depots; publish notices; manage users.
Executive (read-only)	Laptop	See yesterday's report. Maybe download it as a PDF.

You will seed a few users of each role in your dummy data, log in as them, and that's how you'll demo the system.

§4 The Dummy Data Approach (Important)

This is the single biggest scope decision in your project — read this section carefully.

What you'll seed

Write a **seed script** that, on a fresh database, inserts:

- **3–5 depots** in real NCR locations (Noida Sec-37, Anand Vihar, Ghaziabad, etc.) with polygons drawn around them.
- **~50 vehicles**, spread across depots, with realistic registration numbers.

- **~80 users**: a handful of admins, managers, control-room operators; the rest drivers and conductors. All with simple, known passwords (`driver1 / password` etc.) — clearly marked "demo only".
- **10–15 routes** connecting real-ish stops in NCR, each with 6–15 stops and planned timings.
- **One week of duties** — every depot has a published roster covering yesterday, today, tomorrow.
- **One day of GPS history** — for at least 10 vehicles, the entire ping trail of one trip from yesterday. This is what makes "view past trip" demo-able.
- **A handful of incidents** in different states (open, in progress, resolved) so the IMS list isn't empty.
- **2–3 notices** in CMS — one acknowledged by some drivers, one not.

> **TIP**
> Use a library like `faker` (Python) or `@faker-js/faker` (Node) for names, registration numbers, and phone numbers. For GPS history, pick a real route on a map, click 30 points along it, save the coords, then have the seed script step through them with realistic timestamps.

Making the map "live"

Two options — pick one:

1. **The tick script (recommended).** A small Python/Node script runs in the background. Every 5–10 seconds, it picks each "active" vehicle and inserts one new GPS row that's the next plausible position along its current trip. The dashboard polls or websockets every few seconds and re-renders. Simple, demo-friendly.
2. **The replay mode.** The "live" map is actually replaying yesterday's GPS data at 1× or 5× speed, with a clock at the top of the screen showing "demo time". Fine for the demo, less honest about what the system would do in production. Acceptable.

What this approach saves you from

YOU DON'T NEED TO BUILD	BECAUSE
A TCP socket server for device protocols	The tick script writes straight to the DB through your API.
A message queue / DLQ / dead-letter handling	One database insert per tick. Done.
Idempotency keys, deduplication logic	You control both producer and consumer.
Backpressure handling	10 inserts per second is not pressure.
A separate ingest service / worker process	Everything is in one backend app.

> **BE HONEST ABOUT THIS**
> In your final presentation, say clearly: "We seeded the data and animated the live map with a tick script. In a production system, this would be a TCP ingest service feeding a queue feeding a processor — here is what that would look like." Show a one-slide diagram of the production shape. That earns you full credit. Pretending you built the production version when you didn't, loses credit.

The Four Modules

Module 1 — AVLS (Live Map & History)

Show me where every bus is, and let me see where it was.

Must have:

- A map page (use Leaflet — it's free, easy, well-documented).
- One marker per active vehicle, with a tooltip showing the vehicle number and current speed.
- Markers update every 5–10 seconds (your tick script makes this happen).
- Filter by depot (a dropdown is fine).
- Click a marker → side panel shows: vehicle reg no, current driver, current route, last 30 minutes of pings drawn as a polyline.
- A "history" page: pick a vehicle + a date → draw the day's path on the map.

Nice to have (stretch):

- Replay slider on the history page (play / pause / 5× speed).
- Color the marker by status (on-time green, late amber, idle grey).
- A "geofence breach" alert — when a vehicle's last ping is outside the depot polygon when it should be inside, show a banner.

What this teaches: working with geographic data, polling vs websockets, rendering 100+ markers efficiently, time-range queries on a database table.

Module 2 — Scheduling (Routes & Duties)

Tell me who's driving what, when.

Must have:

- Admin can create/edit/delete **routes**: name, depot, ordered list of stops with planned arrival times.
- Depot manager sees a **roster grid** for the week: rows = drivers, columns = days, cells = assigned vehicle + route. Empty cells are unassigned.
- Manager can **assign** a driver to a vehicle for a day (one form).
- Manager clicks **Publish** → assigned drivers can see their duty in the driver app.
- Driver app shows **today's duty**: depot, vehicle, route, start time.
- Driver taps **Acknowledge** → recorded with timestamp.

Nice to have:

- A check that prevents assigning the same driver to two duties on the same day.
- A "copy last week's roster" button.
- Driver can request leave; manager approves.

What this teaches: CRUD UIs that don't suck, modelling relationships (driver ↔ vehicle ↔ route ↔ date), publish/draft workflows.

Module 3 — IMS (Incidents)

Something went wrong. Track it from "open" to "resolved" so it doesn't get forgotten.

Must have:

- **Raise incident** form: type (dropdown — breakdown / accident / complaint / other), severity (P1/P2/P3), description, optional vehicle link, optional photo upload.
- Driver app has a **panic button**: one big red button that auto-creates a P1 incident with the driver's current vehicle and (if you can get it) location.
- **Incident list** with filters: status, severity, depot, mine-only.
- Incident **detail page**: full info, timeline of status changes, free-text notes.
- **Status workflow**: Open → Acknowledged → In Progress → Resolved → Closed. Each change requires a note.
- Manager can **assign** an incident to someone (themselves or another user).

Nice to have:

- SLA timer per severity (P1 = 1 hour, P2 = 4 hours, P3 = 24 hours). Show "time remaining" in the list. Highlight breaches in red.
- An incident linked to a vehicle has a button "Show this vehicle's path around incident time" → jumps to AVLS history.

What this teaches: state machines in real software, audit trails, list-with-filters UI patterns.

Module 4 — CMS (Notices)

Publish a message; track who has read it.

Must have:

- Admin creates a **notice**: title, body (markdown or plain text), target audience (all drivers / specific depot / specific role), publish date.
- Driver app shows **unread notices** on opening; tapping one marks it read.
- Admin can see the **read receipt list**: which drivers have/haven't read a given notice.

Nice to have:

- "Acknowledgement required" notices — driver has to tap a confirm button (not just open).
- PDF attachment.
- Bilingual (English + Hindi) bodies.

What this teaches: targeting/audience rules, simple rich-text handling, file upload to a server.

Plus — Auth and Roles (Not a Module, But Mandatory)

- Login page with username + password. Use **bcrypt** for hashing — never store passwords in plain text, even in a school project.
- JWT tokens for the session.
- Five roles: `driver`, `conductor`, `depot_manager`, `control_operator`, `admin`.
- Every API route checks: "is the user logged in? do they have the role allowed for this action?"
- Depot managers and drivers see only their own depot's data. Control operators and admins see everything.

§5 Recommended Tech Stack (Pick This Unless You Have a Reason)

You can use other tools, but if you're not sure what to pick, use exactly this. It's beginner-friendly, well-documented, and free.

BACKEND LANGUAGE & FRAMEWORK

Python + FastAPI

Auto-generated API docs at `/docs`, great type hints, async-friendly, easy to learn. Alternative: Node.js + Express if your team is stronger in JavaScript.

DATABASE

PostgreSQL + PostGIS

One database for everything. PostGIS gives you map polygons and distance queries. Use SQLite for local dev if Postgres is annoying to install.

ORM / DB ACCESS

SQLAlchemy + Alembic (Python) or Prisma (Node)

An ORM saves you from string-concatenated SQL (which is also how injection attacks happen). Alembic/Prisma migrate handle versioned schema changes.

FRONTEND

React + Vite

Vite for fast dev server. React because you probably know it already. Add TypeScript if your team is comfortable — it pays back in week two.

MAPS

Leaflet + OpenStreetMap tiles

Free, simple API, plays well with React via `react-leaflet`. No API keys needed for OSM tiles in development.

DRIVER "APP"

Mobile-friendly web (PWA) — same React app

Don't build a separate React Native app for the capstone. A responsive React route at `/driver` with a manifest and service worker is plenty.

AUTH

FastAPI's OAuth2 password flow + JWT, or NextAuth/Auth.js for Node

Use the framework's built-in patterns; don't roll your own. Bcrypt for password hashing.

CONTAINERS

Docker + Docker Compose

One `docker compose up` brings up the DB, backend, and frontend. This is your demo-day insurance.

LIVE UPDATES

Polling first (every 5s), WebSocket later

Polling is 10 lines of code and works fine for 100 vehicles. Add WebSocket only if you have time and want the stretch credit.

TESTS & CI

Pytest / Vitest + GitHub Actions

Free for student accounts. Set up CI on day three — not week 20.

IF YOU MUST DEVIATE

Write one paragraph in your README explaining what you chose instead and why. That's it. Evaluators care about the reasoning, not the brand.

§6 A Simple Data Model

Eleven tables. Start with these; add more only if a feature absolutely needs one.

```
depot          (id, code, name, location_lat, location_lng, polygon_geojson)
vehicle        (id, reg_no, depot_id, status)
user           (id, username, password_hash, full_name, role, depot_id, phone)
stop           (id, name, lat, lng)
route          (id, code, name, depot_id)
route_stop     (route_id, stop_id, sequence, planned_offset_min)
duty           (id, date, vehicle_id, driver_id, conductor_id, route_id,
                start_time, end_time, status, ack_at)
gps_ping       (id, vehicle_id, ts, lat, lng, speed_kmh, ignition_on)
incident       (id, type, severity, status, raised_by, assigned_to,
                vehicle_id, depot_id, description, created_at, resolved_at)
incident_event (id, incident_id, ts, actor_id, from_status, to_status, note)
notice         (id, title, body, audience_json, publish_at, created_by)
notice_read    (notice_id, user_id, read_at)
```

The four indexes you actually need

- `gps_ping (vehicle_id, ts DESC)` — for "last N pings of this vehicle".
- `duty (date, depot_id)` — for the roster view.
- `incident (status, depot_id)` — for the incident list.
- A spatial index on `depot.polygon_geojson` — only if you implement geofence detection.

> **TIP**
> Don't try to model everything before you write code. Build the simplest version, run the seed script, build the AVLS map, *then* realise what you forgot and add it. Migrations are made for this.

§7 Quality Bar (Reasonable, Not Heroic)

- **Speed.** Pages load in under 3 seconds on a normal laptop. API responses in under 500 ms for normal queries.
- **Scale.** The demo runs with 50 vehicles and 80 users seeded. Should keep working at 500 vehicles. Don't worry beyond that.
- **Reliability.** If you submit a form successfully, the data is saved. If you click Publish, the duty really is published. No silent failures.
- **Security basics.** Passwords hashed (bcrypt). Auth required on every API route except login. Users can't see other depots' data unless their role allows it. Don't commit secrets to git — use a `.env` file with `.env.example` in the repo.
- **Mobile-friendly.** The driver routes work on a phone screen (320 px wide). The control-room map and admin pages can be desktop-only.
- **Demoable.** `docker compose up` from a fresh clone brings up the whole thing and the seed runs automatically.

§8 What to Hand In

#	DELIVERABLE	FORMAT
1	Source code	Git repo (GitHub, share with evaluators)
2	README	Markdown — quickstart, demo credentials, screenshots
3	Seed script	Part of the repo; runs automatically on `docker compose up`
4	Architecture note	3–5 page document or markdown file — diagram + how each module is built + what you'd improve
5	API documentation	Auto-generated at `/docs` (FastAPI gives you this free; Express needs Swagger setup)
6	A few tests	Doesn't have to be comprehensive — a handful of meaningful tests beats a hundred trivial ones
7	Demo video	10 minutes, shows the live demo flow below
8	Final presentation	10–15 slides, defended live

The demo flow we want to see

1. Run `docker compose up` on a clean machine — everything starts, seed runs.
2. Log in as an **admin**; show the dashboard, create a new route.

3. Log in as a **depot manager**; show the roster grid, publish tomorrow's duties.
4. Log in as a **driver** on a phone-sized window; see today's duty, acknowledge it, open the latest notice, tap the panic button → P1 incident created.
5. Log in as the **control-room operator**; show the live map updating; click a marker; see the new P1 in the incident queue; assign it.
6. Open a past trip in the history view; show the path on the map.
7. (If time): show the API docs at `/docs`.

If you can run that demo without a hitch, the project is a success.

§9 How You'll Be Graded

CATEGORY	WHAT IT MEASURES	WEIGHT
Functional completeness	How many "must have" items across the four modules actually work	40%
Code quality & structure	Readable code, clean module boundaries, sensible naming, no copy-paste forests	15%
UI / UX	Pages are usable, navigation is clear, errors are explained, mobile screens work	15%
Documentation	README that works on day one, architecture note that explains the system	10%
Demo & presentation	You can run the demo, you can explain how it works, you can answer "why did you choose X"	15%
Honesty & engineering judgment	Being clear about what's seeded vs simulated; flagging known limitations	5%
STRETCH CREDIT (UP TO +10%)		
+ WebSocket live updates instead of polling		+3%
+ Geofence breach detection working		+3%
+ SLA timers + breach alerts in IMS		+2%
+ Trip replay with scrubber on history page		+2%

§10 How to Actually Start (Week-1 Tips)

1. **Get the boring stuff out of the way first.** One person sets up the repo, Docker Compose with Postgres + backend + frontend, basic README. Until `docker compose up` works, no feature work.
2. **Write the data model in code, not in a doc.** Create the ORM models from §6, run the first migration, write the seed script with 2 depots, 5 vehicles, 5 users. Confirm you can SELECT them.
3. **Build login + role check before anything else.** Every module assumes auth works. Get it working day one.
4. **Pick the easiest module first — CMS.** Notices are just CRUD with a target audience. Build it end-to-end (DB → API → admin page → driver page). You'll learn your stack while building something you can actually demo. *Then* tackle the bigger modules.
5. **Build the AVLS map early, even with one fake vehicle.** Maps are the most demo-worthy part. Get a Leaflet map showing one marker that moves when you reload — that's a huge motivator for the team.
6. **Divide work by module, not by layer.** One person doing all backend and another doing all frontend is a recipe for "they're blocked on me" all semester. Two pairs of one-backend-one-frontend, each owning a couple of modules end-to-end, works better.
7. **Demo something every two weeks** — to each other, to your guide. Things you don't demo, you don't really have.

> AVOID THESE MISTAKES
>
> - **Don't** spend three weeks designing the perfect data model before writing code. Pick a reasonable one and refactor as you learn.
> - **Don't** build a microservices architecture. One backend app with modules is correct for this project.
> - **Don't** store passwords as plain text "just for now". You will forget to fix it.
> - **Don't** commit your `.env` file or any secrets to git. Use `.gitignore` from commit one.
> - **Don't** wait until the last week to test the demo flow. Rehearse it from month two onwards.

§11 Where to Get Help

Read these once, early

- FastAPI tutorial (or the Express docs if you went Node) — work through the whole tutorial. It will save you a month.
- PostgreSQL tutorial — at least chapters 1–4.
- Leaflet quick-start.
- OWASP Top 10 — read it once, just to know what kinds of mistakes you're avoiding.
- The 12-Factor App — ten-minute read; tells you how to structure a real app.

Read these when you get stuck

- **"How to set up Docker Compose for FastAPI + Postgres"** — search; pick a recent tutorial.
- **"react-leaflet markers update performance"** — when your map gets slow with many markers.
- **"FastAPI dependency injection auth"** — for clean role-check decorators.
- **PostGIS docs on `ST_Contains`** — for "is this point inside that polygon".

Using AI tools

Yes — Copilot, Claude, ChatGPT are fine and encouraged. But: every line in your repo, you must be able to explain in your demo. If we ask "why did you write it this way" and the answer is "the AI wrote it", credit goes down. Treat AI output as a first draft to learn from, not as the final answer.

§12 Final Words

This project is a real one. NCRTC runs bus operations for crores of commuters in NCR, and the software shape you're building is what real fleet platforms look like under the hood. You won't take it to production — but you will, after building this, understand exactly what your first job at a startup or a service company involves.

The single best advice we can give you is: **build small and build often**. Ship the ugly version of every feature first. Make it work. Then make it nice. Don't aim for the perfect cathedral — aim for a working bus depot.

Good luck. Have fun.